Pestaña 1

Infraestructura de Servicios

PROF. ROQUE TRUJILLO RAMOS

Practica de Kubernetes

UNIDAD 2

Ing.en Sistemas Computacionales

Realizado por:
Talavera Uribe Jose Miguel

Control del informe

Proyecto: [nombre aprobado] | Repositorio: [URL] | Etiqueta/commit de entrega: [versión]

Inicio: [fecha] | Entrega: [fecha] | Versión del informe: [0.1] | Estado: plantilla por completar.

Control de cambios: [versión, fecha, autor, cambio y revisión].

Completar los campos entre corchetes con datos propios. Adjuntar archivos y mediciones reales; conservar pruebas fallidas y sus repeticiones. No incluir secretos ni datos personales reales.

Guía de uso: los capítulos D01–D10 integran el expediente técnico; los encabezados permiten navegar desde el esquema de Google Docs. Al finalizar, generar el índice y revisar su paginación.

Plan de trabajo: [actividad, fecha prevista, fecha real y entregable]. Confirmar con el docente el calendario: el PDF menciona dos semanas de duración sugerida y también presenta un calendario de ocho semanas.

D01. Ficha del proyecto y requerimientos

1.1 Resumen y ficha de aprobación

Proyecto y propósito. Tienda de ropa alternativa, desarrollada por Talavera Uribe Jose Miguel para la práctica individual de Infraestructura de Servicios. Se adapta el sistema existente en /sistema y se conserva su catálogo, estilo visual, carrito y ventas. El objetivo es controlar compras con reserva temporal y preparar su operación en ambientes Kubernetes separados.

Problema y valor. Una prenda puede tener existencias en una talla y estar agotada en otra. Si varios clientes intentan comprar las últimas unidades o dejan pedidos sin pagar, la tienda necesita evitar la sobreventa y recuperar las reservas abandonadas. La solución asigna un plazo de 30 minutos para pagar, muestra el estado del pedido y conserva el historial de movimientos.

Usuarios. El cliente consulta prendas y tallas, administra su carrito, crea pedidos, realiza pagos simulados y consulta sus compras. Puede cancelar únicamente sus pedidos pendientes de pago. El administrador gestiona productos, tallas, inventario y usuarios; prepara pedidos confirmados y registra su entrega. Comprar una edición limitada no requiere autorización administrativa.

Alcance. Se incluyen catálogo, autenticación, dos roles, carrito, pedidos por talla, reservas, pago simulado, cancelación, vencimiento, búsqueda, paginación, reporte de ventas e historial consultable. Se excluyen pagos bancarios reales, fabricación o personalización, facturación fiscal e integración con paqueterías. Se utilizarán datos ficticios.

RN01 Reserva por variante. El carrito no aparta unidades. Al crear un pedido se verifica la existencia por prenda y talla y se descuenta su reserva en una transacción. Si falta una variante, se rechaza la compra completa. Validación: dos compradores concurrentes no pueden reservar la misma última unidad.

RN02 Plazo de pago. El cliente dispone de 30 minutos desde la creación del pedido. Un pago simulado válido dentro del plazo lo confirma automáticamente. Si no paga, el pedido vence y pierde la reserva; las prendas quedan disponibles para otros compradores. Validación: se rechaza pagar después del vencimiento, incluso si la tarea programada aún no ha corrido.

RN03 Ediciones limitadas. Cada cliente puede adquirir como máximo dos unidades del mismo modelo marcado como edición limitada, sumando tallas y pedidos vigentes o entregados. Se excluyen cancelados y vencidos. Validación: dividir la compra en varios pedidos no evita el límite.

RN04 Cancelación y trazabilidad. Cancelar o vencer devuelve las unidades una sola vez. Los pedidos confirmados ya no pueden cancelarse desde la cuenta del cliente. Se conservan precio, nombre y talla comprados aunque cambie el catálogo. Validación: repetir una operación no duplica pedidos, pagos ni liberaciones.

Módulos y tecnologías. Se conservan autenticación, usuarios, productos, tallas, carrito, pedidos y ventas. Se incorporan reservas, estados, tareas de vencimiento e historial. La interfaz utiliza HTML, CSS y JavaScript; el backend modular utiliza PHP y CodeIgniter 3.1.13. Docker empaqueta Nginx y PHP-FPM; Kubernetes administra las cargas, servicios, configuración y almacenamiento. La imagen del backend tiene PHP 8.3 como versión objetivo.

Base relacional. PostgreSQL es la fuente de verdad para usuarios, roles, prendas, tallas, existencias, pedidos, detalles y reservas. Las relaciones y transacciones protegen inventario y compras. Las sesiones compartidas permiten atender al mismo cliente desde distintas réplicas del backend.

Base no relacional. MongoDB mantiene el historial visible de reservas, pagos, estados, cancelaciones y vencimientos. Los eventos tienen datos variables, como tallas y cantidades liberadas o motivos del cambio. Su modelo documental permite reunir esos detalles sin una columna por cada tipo de evento; el cliente consulta la línea de tiempo y el administrador revisa movimientos de inventario.

Consistencia. El cambio principal y un evento pendiente se guardan juntos en PostgreSQL mediante outbox. Un proceso lo envía a MongoDB con identificador único y escritura idempotente. Si la segunda base falla, la compra se conserva y el historial muestra sincronización pendiente. La recuperación sin duplicados debe comprobarse con MongoDB operativo.

Flujo. Carrito → Pendiente de pago → Confirmado → En preparación → Listo para entrega → Entregado. El carrito precede al pedido; desde Pendiente de pago también se permite Cancelado o Vencido. El cliente paga o cancela, el sistema confirma o vence y el administrador prepara y entrega. Se rechazan saltos de etapa.

Diferenciación y registro. Las reservas por talla con vencimiento, el límite acumulado por cliente y la trazabilidad de inventario modifican el funcionamiento del sistema. La propuesta conserva el negocio de ropa alternativa. La aprobación y el registro de no duplicidad ante el docente están pendientes; no se presenta una aprobación que todavía no está documentada.

1.2 Requerimientos funcionales

RF01 Interfaz y comunicación. La tienda deberá conservar su apariencia de ropa alternativa, ofrecer catálogo por talla, carrito y pantalla de pedidos con mensajes claros. Aceptación: navegar desde un teléfono y un escritorio, generar un pedido y visualizar su plazo sin desbordamientos. Verificación: tests/http.php cubre acceso al carrito y a pedidos; falta anexar revisión visual adaptable. Estado: implementación disponible y validación visual pendiente.

RF02 Backend y acceso a datos. Las solicitudes de compra se procesarán en el servidor; el navegador no se conectará a PostgreSQL ni a MongoDB. Aceptación: los importes se calcularán con precios del catálogo y las credenciales no aparecerán en el contenido entregado al cliente. Verificación: tests/domain.php comprueba que se ignore el precio enviado por el cliente. Estado: regla implementada; falta evidencia de red y revisión del navegador.

RF03 Entidades relacionadas y bajas. Se administrarán productos, usuarios y pedidos; cada pedido pertenecerá a un usuario y sus detalles relacionarán producto y talla. Aceptación: crear, consultar y modificar registros según su rol; dar de baja productos y usuarios de forma lógica; cancelar pedidos sin borrar su historial. Verificación: controladores y modelos del repositorio y prueba manual completa de CRUD. Estado: operaciones disponibles; validación integral pendiente.

RF04 Autenticación y autorización. Existirán los roles cliente y administrador, con validación en servidor. Aceptación: un cliente no podrá consultar ni pagar pedidos ajenos ni avanzar etapas administrativas. Verificación: tests/domain.php prueba acceso ajeno y restricciones de estados; tests/http.php prueba login y sesión. Estado: comprobaciones locales parciales aprobadas; falta anexar matriz completa de permisos.

RF05 Validación de entradas. Se validarán campos obligatorios, correo, contraseñas, identificadores, cantidades positivas, tallas, existencias y relaciones. Aceptación: cantidades cero, variantes inexistentes o compras sin stock serán rechazadas sin cambios parciales; las reglas se mantendrán al omitir la interfaz. Verificación: tests/domain.php y pruebas negativas sobre formularios y endpoints. Estado: validaciones de compra probadas; revisión general de formularios pendiente.

RF06 Consultas y reporte. Se ofrecerán búsqueda y paginación de pedidos y un reporte de ventas con prenda, talla, cantidad, importe y estado. Aceptación: filtrar por número o estado, navegar páginas y excluir del importe de ventas los pedidos sin pagar, cancelados o vencidos. Verificación: pantalla de pedidos y modelo Venta_model.php; comprobar totales con un conjunto conocido. Estado: código disponible; prueba del reporte y evidencia pendientes.

RF07 Reserva y flujo de compra. Se aplicarán RN01 a RN04 y el flujo de estados descrito en la ficha. Aceptación: reservar por 30 minutos, confirmar el pago simulado sin aprobación administrativa, liberar reservas vencidas y permitir solo transiciones válidas. Verificación: tests/domain.php cubre concurrencia, idempotencia y pago tardío; tests/http.php cubre compra y pago. Estado: verificado localmente; falta comprobar la ejecución programada en Kubernetes.

RF08 Datos de prueba reproducibles. Una semilla idempotente generará al menos 100 prendas de negocio con variantes por talla y cuentas ficticias de ambos roles. Aceptación: ejecutarla dos veces sin duplicar prendas ni restablecer las existencias ya modificadas. Verificación: database/seed.php y conteos antes y después en un ambiente aislado. Estado: semilla implementada; ejecución y evidencia de idempotencia pendientes.

RF09 Historial y recuperación entre bases. La aplicación mostrará eventos de MongoDB y conservará en PostgreSQL los envíos pendientes si la segunda base no responde. Aceptación: crear un pedido durante una caída de MongoDB, restablecer el motor, sincronizar y repetir el envío sin eventos duplicados. Verificación: TiendaHistory.php, outbox y tests/domain.php; el flujo HTTP comprueba consulta degradada. Estado: conservación de pendientes probada localmente; sincronización real y recuperación sin duplicados pendientes.

1.3 Requerimientos no funcionales

Los siguientes valores son objetivos iniciales de aceptación del laboratorio. Deberán medirse y ajustarse con evidencia; no representan resultados ya obtenidos.

RNF01 Disponibilidad de aplicación. Producción tendrá al menos dos réplicas de frontend y backend distribuidas entre los dos workers. Objetivo: al eliminar un Pod de aplicación, conservar al menos una respuesta válida por capa y registrar interrupción, errores y tiempo de recuperación durante una observación de cinco minutos. Evidencia prevista: monitoreo, solicitudes repetidas y estado de Pods. Estado: manifiestos preparados; prueba de clúster pendiente.

RNF02 Rendimiento. Objetivo inicial: latencia p95 de consultas de catálogo y pedidos menor o igual a 800 ms y menos de 1 % de respuestas 5xx durante cinco minutos con 50 usuarios virtuales y datos semilla. Se documentarán recursos, tasa de solicitudes y calentamiento. Evidencia prevista: informe de carga y métricas por capa. Estado: pendiente de medición.

RNF03 Seguridad. Objetivo: 100 % de los casos negativos de autorización definidos deberán ser rechazados; todo POST de la interfaz requerirá protección CSRF y las contraseñas se almacenarán con hash. Ambos ambientes usarán HTTPS; producción también verificará TLS hacia los motores, mínimos privilegios y políticas de red. Evidencia prevista: pruebas de rol, sesión, certificado, usuario de base y conexiones permitidas o bloqueadas. Estado: controles de aplicación parciales implementados; seguridad de infraestructura y TLS de bases pendientes.

RNF04 Persistencia y recuperación. Objetivos iniciales: RPO máximo de 24 horas y RTO máximo de 30 minutos para cada motor. La restauración se realizará en un espacio aislado, comprobando conteos, relaciones, índices y operación de la tienda. Evidencia prevista: respaldos, registros y cronometraje. Estado: respaldo local previo disponible; estrategia programada de ambos motores y restauración pendientes.

RNF05 Escalado y capacidad. El backend productivo tendrá HPA de 2 a 6 réplicas y objetivo de CPU del 70 %. Se registrarán requests, limits, CPU, memoria y comportamiento si faltan recursos. Aceptación: con carga sostenida se observará la decisión del HPA, la

creación de réplicas y su reducción posterior sin errores de inventario. Estado: configuración preparada; métricas y ensayo pendientes.

RNF06 Reproducibilidad y aislamiento. Se mantendrán dos clústeres con contextos, credenciales, datos y volúmenes independientes. Aceptación: reconstruir cada ambiente desde los archivos versionados y el procedimiento, sin secretos en Git ni pasos tácitos. Se registrarán versiones y digests de las imágenes. Evidencia prevista: ejecución del manual y revisión de recursos. Estado: archivos de despliegue disponibles; reconstrucción completa pendiente.

RNF07 Integridad y sincronización. Objetivo: cero sobreventas y cero movimientos duplicados en los casos de concurrencia y reintento. El pago tardío será rechazado en el servidor; la liberación programada tendrá como objetivo ejecutarse dentro de los 60 segundos posteriores al plazo, con el clúster sano. Tras recuperar MongoDB se verificará que cada evento aparezca una sola vez. Estado: integridad local probada; tiempos del CronJob y recuperación de MongoDB pendientes.

RNF08 Observabilidad. Se deberán consultar métricas y logs de frontend, backend, nodos y ambos motores; al menos dos alertas se activarán y resolverán mediante pruebas controladas. Aceptación: correlacionar un fallo con el pedido o servicio afectado y conservar la evidencia. Estado: implementación y validación de la plataforma de observabilidad pendientes.

D02. Investigación y decisiones de arquitectura

2.1 Investigación aplicada

[Explicar cada concepto con fuente oficial, versión, fecha de consulta, ejemplo propio y recurso/archivo del proyecto.]

Temas: plano de control y nodos; Pod y Deployment; Service y DNS; CNI y NetworkPolicy; CSI y almacenamiento; StatefulSet; ConfigMap y Secret; RBAC; startup/readiness/liveness; HPA; distribución de réplicas; actualizaciones y recuperación.

2.2 Comparación de alternativas

Distribuciones Kubernetes: [opción A / opción B, versiones, recursos, soporte, seguridad y reproducción]. Selección y justificación: [completar].

Entrada de tráfico: [opción A / opción B, Gateway API o Ingress, controlador mantenido, compatibilidad y mecanismo de IP externa]. Selección: [completar].

2.3 Registro de decisiones

ADR01 Distribución Kubernetes: [decisión, alternativa descartada, motivo, costo, limitación, versión y archivo asociado].

ADR02 CNI y cumplimiento de NetworkPolicy: [decisión, alternativa descartada, motivo, costo, limitación, versión y archivo asociado].

ADR03 Entrada HTTPS y balanceador/IP externa: [decisión, alternativa descartada, motivo, costo, limitación, versión y archivo asociado].

ADR04 Persistencia y almacenamiento: [decisión, alternativa descartada, motivo, costo, limitación, versión y archivo asociado].

ADR05 Motores relacional y no relacional: [decisión, alternativa descartada, motivo, costo, limitación, versión y archivo asociado].

ADR06 Observabilidad: [decisión, alternativa descartada, motivo, costo, limitación, versión y archivo asociado].

D03. Arquitecturas, inventario y capacidad

3.1 Diagramas obligatorios

Figura 1. Aplicación: actores, interfaz, API, módulos, autenticación y ambas bases; protocolos y responsabilidades. [Insertar imagen legible y leyenda; enlazar archivo editable y manifiestos correspondientes.]

Figura 2. Secuencia de un CRUD, incluyendo validación, autorización y errores. [Insertar imagen legible y leyenda; enlazar archivo editable y manifiestos correspondientes.]

Figura 3. Secuencia entre ambos motores: fuente de verdad, fallo, reintento y recuperación sin duplicados. [Insertar imagen legible y leyenda; enlazar archivo editable y manifiestos correspondientes.]

Figura 4. Despliegue de desarrollo: máquinas, roles, namespaces, cargas, Services, réplicas, HTTPS, PVC, StorageClass y observabilidad. [Insertar imagen legible y leyenda; enlazar archivo editable y manifiestos correspondientes.]

Figura 5. Despliegue de producción: ubicación de componentes, réplicas entre workers y dependencias de almacenamiento. [Insertar imagen legible y leyenda; enlazar archivo editable y manifiestos correspondientes.]

Figura 6. Comunicación: red, gateway, firewall, administración, IP de nodos, CIDR de Pods/Services, DNS, balanceador, Internet y VLAN/NAT/VPN si aplica. [Insertar imagen legible y leyenda; enlazar archivo editable y manifiestos correspondientes.]

3.2 Inventario y presupuesto

Por máquina: [ambiente, nombre, rol, tipo físico/virtual, CPU/arquitectura, RAM, disco, SO/versión, IP y capacidad disponible]. Añadir consumo medido de aplicación, bases, CNI y observabilidad.

Desarrollo: mínimo un nodo y una réplica inicial por capa. Producción: mínimo un control plane y dos workers; al menos dos réplicas de frontend y backend.

Separación verificable: [contextos, credenciales, configuración, datos, volúmenes y nombres HTTPS distintos]. Dos namespaces no sustituyen dos clústeres.

Límites: [control plane único, balanceador único, bases/volúmenes sin redundancia, host compartido y efecto de agotamiento de capacidad]. Ampliaciones de alta disponibilidad: [diseño, alcance implementado y pendiente].

D04. Modelos de datos y consistencia

4.1 Base relacional

Motor/versión y justificación: [completar]. [Insertar diagrama ER y enlace editable].

Diccionario por entidad: [campo, tipo, PK/FK, nulabilidad, restricción, índice y descripción]. Transacciones y consultas representativas: [archivo y explicación].

Migraciones y semilla: [rutas, orden, comandos y validación]. Gestión de conexiones con varias réplicas: [pool, límites y presupuesto total].

4.2 Base no relacional

Motor/versión, función visible y valor de su modelo: [completar; una caché sola no cubre el requisito].

Modelo/diccionario: [colección/estructura, campos, tipos, validaciones, índices, consultas y ejemplo sintético]. Scripts idempotentes: [rutas y ejecución].

4.3 Consistencia y recuperación

Fuente de verdad: [por dato]. Estrategia: [outbox, reintento idempotente o equivalente; claves de deduplicación y reconciliación].

Prueba de fallo de escritura secundaria: [precondición, acción, resultado, retraso/pérdida observada, recuperación y verificación sin duplicados]. Evidencia: [ID].

D05. Manual de instalación desde cero

5.1 Versiones, prerrequisitos y repositorio

Inventario de software: [componente, versión/digest, dependencia, origen oficial, compatibilidad y archivo de configuración].

Mapa del repositorio: [rutas de frontend, backend, pruebas, infraestructura base, desarrollo, producción, observabilidad, red, respaldos y documentación]. Incluir Dockerfiles, dependencias fijadas, charts/manifiestos, scripts, migraciones, semillas y README.

Automatización: [construcción, pruebas, validación de manifiestos y ejecución]. Exclusiones de Git: [secretos, kubeconfig privilegiado y dumps sensibles].

5.2 Ficha reutilizable de cada paso

Paso [número y objetivo] | Ambiente/contexto: [completar].

Prerrequisitos: [equipos, permisos, dependencias y estado previo]. Archivo: [ruta y versión].

Comando exacto sin secretos: [completar]. Parámetros: [explicar].

Resultado esperado: [completar]. Comprobación de éxito: [comando y condición]. Evidencia: [ID y vínculo].

Error posible/diagnóstico: [completar]. Solución: [completar]. Reversión cuando corresponda: [completar].

5.3. Instalación de desarrollo

[Completar ficha de paso] Preparar equipos: nombres/IP, DNS, reloj, SO, CPU e imágenes.

[Completar ficha de paso] Crear clúster: instalación, roles, contexto y nodos Ready.

[Completar ficha de paso] Habilitar plataforma: CNI, DNS, CSI/StorageClass, controlador de entrada, IP externa y API de métricas.

[Completar ficha de paso] Aplicar controles: namespaces, cuotas, LimitRange, RBAC, seguridad, secretos y políticas de red.

[Completar ficha de paso] Desplegar datos: PVC, ambos motores, TLS según ambiente, usuarios, índices, semilla y Job único de migración.

[Completar ficha de paso] Desplegar/publicar aplicación: imágenes, configuración, sondas, recursos, Services y HTTPS.

[Completar ficha de paso] Validar ambiente: salud, rutas HTTPS, independencia de datos y comprobaciones funcionales.

5.4. Instalación de producción de laboratorio

[Completar ficha de paso] Preparar equipos: nombres/IP, DNS, reloj, SO, CPU e imágenes.

[Completar ficha de paso] Crear clúster: instalación, roles, contexto y nodos Ready.

[Completar ficha de paso] Habilitar plataforma: CNI, DNS, CSI/StorageClass, controlador de entrada, IP externa y API de métricas.

[Completar ficha de paso] Aplicar controles: namespaces, cuotas, LimitRange, RBAC, seguridad, secretos y políticas de red.

[Completar ficha de paso] Desplegar datos: PVC, ambos motores, TLS según ambiente, usuarios, índices, semilla y Job único de migración.

[Completar ficha de paso] Desplegar/publicar aplicación: imágenes, configuración, sondas, recursos, Services y HTTPS.

[Completar ficha de paso] Validar ambiente: salud, rutas HTTPS, independencia de datos y comprobaciones funcionales.

5.5 Reconstrucción en recursos limpios

[Inventario limpio, versión usada, ejecución completa del manual, duración, diferencias, pasos corregidos y evidencia E14]. No dejar pasos tácitos.

D06. Plan y configuración de seguridad

6.1 Matriz de flujos y firewall

Por flujo: [origen/identidad, destino, protocolo/puerto real, condición, control, orden de regla, motivo, archivo exportable y prueba]. Incluir usuario→HTTPS, administración→API/SSH, API→ambas bases, DNS TCP/UDP, monitorización, respaldos y comunicación entre nodos según distribución/CNI.

Default deny de entrada y salida: [manifiestos]. Permisos explícitos de DNS y dependencias: [archivos]. Bases sin exposición pública/NodePort/entrada web; paneles y administración restringidos: [configuración].

Validación: [Pod permitido y no autorizado contra el mismo destino sano; ingreso y egreso por separado; DNS funcional; administración bloqueada desde origen no autorizado]. Alcance de firewall frente a NetworkPolicy y hostNetwork: [explicar].

6.2 Seguridad por capa

Aplicación/API: [hash de contraseñas, autorización servidor, ORM/consultas parametrizadas, errores seguros, límite de intentos, cookies, CSRF y CORS si aplica].

Datos: [cuenta de migración separada, usuarios de ejecución y monitorización con mínimo privilegio, permisos comprobados].

Contenedores: [no root, sin privilegios/escalamiento, capacidades reducidas, seccomp RuntimeDefault y filesystem de solo lectura cuando viable].

Kubernetes: [matriz identidad/rol/verbo/recurso/namespace, ServiceAccounts, tokens desactivados cuando no se necesitan, PSA Restricted para cargas propias y excepciones mínimas de plataforma].

Secretos: [creación, rotación, recuperación, cifrado en reposo según distribución y protección de kubeconfig]. Base64 no equivale a cifrado.

TLS: [dominios, emisión/confianza/renovación, verificación del certificado; TLS a ambos motores en producción y validación de identidad con CA].

Suministro e infraestructura: [versiones fijadas, imágenes por digest/etiqueta inmutable, escaneo y tratamiento de hallazgos, actualizaciones, auditoría y acceso administrativo].

6.3 Riesgos y excepciones

Por riesgo: [ID, impacto, probabilidad, mitigación, responsable, estado y justificación de excepción]. Vincular pruebas de rol, entradas inválidas, red, certificado y permisos de base; evidencias sin contraseñas, tokens o llaves.

D07. Publicación, promoción y cambios

7.1 Publicación y promoción

Desarrollo: [URL HTTPS, contexto, namespace y configuración]. Producción: [URL HTTPS distinta, contexto, namespace y configuración].

Procedimiento: [artefacto probado, digest, criterios de aprobación, configuración del ambiente, despliegue y verificación]. Promover el mismo artefacto aprobado.

7.2 Réplicas, HPA y operación de cambios

Frontend/backend: [mínimo dos réplicas en producción, Services, startup/readiness/liveness, requests/limits medidos, distribución entre workers, rolling update y PDB]. Evitar que liveness provoque reinicios masivos por caída de BD.

Sesiones y archivos: [mecanismo externo; backend sin estado local compartible]. HPA autoscaling/v2: [mínimo 2, máximo 6, CPU inicial 60 %, API de métricas, ajustes justificados y capacidad disponible].

Actualización y rollback: [versión inicial/nueva, comandos, comprobaciones de funcionalidad, tiempos, impacto y reversión]. Migraciones: [compatibilidad o plan explícito; rollback de imagen no revierte datos/esquema].

Administración de accesos: [alta, cambio, revocación, identidades, responsables y comprobación]. Evidencia E12: [vínculo].

D08. Operación, observabilidad e incidentes

8.1 Paneles, métricas y logs

Pila/versión e instalación declarativa: [completar]. Integraciones/exporters de ambas BD y credenciales mínimas: [completar]. Logs centralizados con fecha y correlación: [ruta/configuración].

Paneles: [nodos: estado, CPU, memoria, disco, red/capacidad]; [Kubernetes: reinicios, Pending, Pods Ready, eventos y réplicas]; [app: tráfico, errores, p95 y dependencias]; [ambas BD: salud, conexiones, operaciones/consultas, latencia y capacidad]; [red/TLS]; [respaldos: última ejecución válida, duración, tamaño y estado].

8.2 Alertas

A01 Aplicación indisponible: [consulta, umbral, duración, severidad, acción y archivo].

A02 Nodo no disponible: [consulta, umbral, duración, severidad, acción y archivo].

A03 Tasa elevada de errores: [consulta, umbral, duración, severidad, acción y archivo].

A04 Saturación de recursos: [consulta, umbral, duración, severidad, acción y archivo].

A05 Respaldo fallido o vencido: [consulta, umbral, duración, severidad, acción y archivo].

Activar y resolver de forma controlada al menos dos alertas: [ID, disparador, línea de tiempo, activación/resolución y evidencia E10]. Sin enviar datos a terceros.

8.3 Manual de incidentes

Fallo de Pod backend: [síntoma, diagnóstico, comandos, acciones, tiempos, impacto real, verificación y escalamiento].

Mantenimiento de worker mediante drain/uncordon respetando PDB y capacidad: [síntoma, diagnóstico, comandos, acciones, tiempos, impacto real, verificación y escalamiento].

Indisponibilidad temporal de una base y reconciliación: [síntoma, diagnóstico, comandos, acciones, tiempos, impacto real, verificación y escalamiento].

Fallo de actualización y rollback: [síntoma, diagnóstico, comandos, acciones, tiempos, impacto real, verificación y escalamiento].

D09. Persistencia, respaldos y continuidad

9.1 Almacenamiento por ambiente

Por motor y ambiente: [StatefulSet/operador y versión, StorageClass/CSI, PVC/PV, capacidad, acceso, reclaim policy, rutas lógicas, afinidad y comportamiento al mover a otro nodo]. No compartir credenciales ni volúmenes.

Persistencia comprobada: [conteos/registros antes y después de recrear Pods, condiciones y E05]. Almacenamiento local: [limitación de movilidad y tiempo de recuperación].

Migraciones: [Job único, orden y control de concurrencia]. Replicación/failover de motores: [diseño, primario, consistencia y alcance pendiente]; aumentar réplicas de StatefulSet no configura replicación automáticamente.

9.2 Plan de respaldos

Para cada motor: [método consistente, comando/script, programación, retención, destino separado del volumen original, cifrado, permisos, fecha, tamaño y comprobación de integridad].

9.3 Restauración aislada y objetivos

Objetivos definidos antes de probar: RPO [valor y justificación]; RTO [valor, inicio/fin de medición y justificación]. El PDF propone inicialmente 24 h y 30 min, ajustables.

Por motor: [respaldo elegido, destino aislado, comandos exactos, duración, conteos, registros, relaciones, índices y prueba de aplicación restaurada]. No restaurar sobre el origen de la prueba.

Resultados: RPO real [medición]; RTO real [medición]; cumplimiento [sí/no y explicación]; evidencia E13 [vínculo].

D10. Pruebas, evidencias y cierre

10.1 Registro reutilizable de prueba

ID/requisito: [completar]. Precondiciones y datos: [completar]. Ambiente/contexto/versión: [completar].

Pasos, comandos, parámetros y duración: [completar]. Resultado esperado: [completar]. Mediciones y resultado observado: [completar].

Estado: [pendiente/aprobada/fallida]. Evidencia: [ID/vínculo]. Diagnóstico del fallo, cambio y repetición: [completar; conservar ambas ejecuciones].

10.2 Casos funcionales y seguridad

[Registrar casos válidos, inválidos, duplicados, usuario sin permisos, sesión expirada y dependencia caída]. Probar desde interfaz y API evitando la interfaz para comprobar validación/autorización del servidor.

10.3 Experimento de carga y balanceo

Escenario/transacción representativa con consulta de datos: [completar]. Umbrales previos p95/errores: [valor, carga y ventana]. Equipo/Pod generador, herramienta/versión, script, concurrencia y duración: [completar].

Línea base de cinco minutos: [réplicas, CPU, memoria, p95, solicitudes/s y errores]. Carga progresiva durante al menos diez minutos o lo necesario para observar HPA: [serie y archivos exportados]. Separar recursos del generador.

Serie temporal: [réplicas deseadas/actuales/Ready, crecimiento por encima de dos, reducción y ventana de estabilización]. Comparación con/sin HPA bajo mismo escenario/datos: [mediciones]. Si no escala: [diagnóstico, ajuste y repetición].

Balanceo: [identificador temporal de réplica o correlación de logs; solicitudes en varias réplicas y distribución entre workers]. Retirar identificación pública después de probar.

Análisis: [cumplimiento de umbrales, saturación de BD, conexiones, capacidad y cuello de botella; sin asumir mejora lineal].

10.4 Matriz de aceptación E01–E14

E01. Dos contextos y topologías distintas; nodos y componentes sanos. Estado: [pendiente]. Prueba/archivo/vínculo: [completar].

E02. URLs HTTPS distintas y datos independientes. Estado: [pendiente]. Prueba/archivo/vínculo: [completar].

E03. CRUD de tres entidades, roles y validaciones desde interfaz y API. Estado: [pendiente]. Prueba/archivo/vínculo: [completar].

E04. Uso funcional persistente de ambas bases con consultas trazables. Estado: [pendiente]. Prueba/archivo/vínculo: [completar].

E05. Recreación de Pods de datos sin pérdida en condiciones documentadas. Estado: [pendiente]. Prueba/archivo/vínculo: [completar].

E06. HPA crece por encima del mínimo y disminuye; series y carga adjuntas. Estado: [pendiente]. Prueba/archivo/vínculo: [completar].

E07. Varias réplicas atienden solicitudes y están distribuidas entre workers. Estado: [pendiente]. Prueba/archivo/vínculo: [completar].

E08. Conexiones permitidas/denegadas; DNS y servicios sanos como control. Estado: [pendiente]. Prueba/archivo/vínculo: [completar].

E09. Denegación por rol de aplicación, RBAC y usuario limitado de BD. Estado: [pendiente]. Prueba/archivo/vínculo: [completar].

E10. Paneles de nodos, servicios y ambos motores; dos alertas activadas/resueltas. Estado: [pendiente]. Prueba/archivo/vínculo: [completar].

E11. Fallo de Pod y mantenimiento de worker con tiempos/impacto medidos. Estado: [pendiente]. Prueba/archivo/vínculo: [completar].

E12. Actualización y rollback con versión y funcionalidad verificadas. Estado: [pendiente]. Prueba/archivo/vínculo: [completar].

E13. Respaldo/restauración de ambas bases; RPO y RTO registrados. Estado: [pendiente]. Prueba/archivo/vínculo: [completar].

E14. Reconstrucción desde recursos limpios sin pasos tácitos ni secretos en Git. Estado: [pendiente]. Prueba/archivo/vínculo: [completar].

10.5 Ficha reutilizable de evidencia

ID: [E__ / sub-ID]. Fecha: [completar]. Ambiente: [completar]. Requisito: [completar]. Comando/acción: [sin secretos]. Esperado: [completar]. Observado y explicación: [completar]. Adjuntos: [captura legible, salida de texto, script y/o métricas exportadas].

10.6 Resultados, limitaciones y conclusiones

Resultados frente a requerimientos y objetivos: [mediciones y evidencias]. Limitaciones y puntos únicos de fallo: [completar]. Pendientes/correcciones: [responsable y fecha]. Conclusiones y aprendizaje: [completar].

Revisión de rúbrica (100 puntos): proyecto/aplicación 15; arquitecturas 10; ambientes/declarativo 15; bases/persistencia/restauración 10; balanceo/HPA/carga 15; seguridad 15; observabilidad/fallos 10; documentación/evidencias/defensa 10. Autoevaluación justificada: [completar].

10.7 Referencias

Fuente de requisitos: Practica_Individual_Kubernetes_Desarrollo_Produccion-2.pdf, consulta de referencia del diseño indicada en el PDF: 16/10/2026.

Por fuente técnica: [organización, título, URL oficial, versión aplicable, fecha de consulta y sección donde se usa]. Añadir distribución, CNI, entrada, almacenamiento, ambos motores y herramientas elegidas.

10.8 Anexos y entrega

[Enlaces a diagramas editables, manifiestos/charts, configuración sin secretos, scripts, migraciones, semillas, pruebas, logs y métricas]. Vincular cada anexo con capítulo y versión del repositorio.

Lista final: [informe PDF y editable]; [diagramas editables e imágenes]; [código con etiqueta]; [manual reproducible]; [D01–D10 completos]; [E01–E14 verificadas]; [sin evidencias ajenas ni resultados fabricados].

Preparación de defensa: [recorrido desde URL hasta cada BD]; [reinicio de contenedor vs recreación de Pod vs réplicas vs nodos]; [efecto de fallos de Pod/worker/control plane/almacenamiento]; [prueba de bloqueo por política con destino sano].